# HallSync

## Complete Database Implementation Guide

Lecture Hall Booking Management System

Faculty of Science prototype | University of Kelaniya, Sri Lanka

MySQL 8.0.46 | Database implementation v2 | 11 November 2026

| Tables | Columns | Foreign keys |
|---|---|---|
| 11 | 61 | 14 |

A study guide for understanding the database, running the complete setup, explaining the rules in a presentation, and connecting an object-oriented application later.

Your existing department record is preserved. The main continuation script adds the remaining database objects without dropping the database.

Validation: 100 checks passed against MySQL 8.0.46. This includes independent-connection approval races, approval-versus-closure races, and a hall-lock waiting check.

The scripts are complete in the repository. They create the database objects on your laptop only when you execute them in Workbench.

# 1. Start here: the one-file continuation

You have already created hallsync_db, the DEPARTMENT table and the Industrial Management row. Use database/complete_existing_database.sql to finish that database in one execution.

- Download the latest repository ZIP and extract it to a normal writable folder.

- Open your existing MySQL Workbench connection and open complete_existing_database.sql.

- Enable Stop Script Execution On Errors. Execute the complete file with no text selected.

- If an error appears, stop and send the error message. Do not drop the database or blindly rerun the file.

- Refresh SCHEMAS. The final verification grid should show the expected object counts below.

| Object | Expected |
|---|---|
| Base tables | 11 |
| Columns in base tables | 61 |
| Primary keys | 11 |
| Foreign keys | 14 |
| Additional unique keys | 7 |
| CHECK constraints | 28 |
| Views | 4 |
| Procedures | 25 (19 public + 6 internal) |

The preflight requires exactly the department table from step 02. It refuses to continue over a partially completed or different schema. This is deliberate: CREATE TABLE IF NOT EXISTS would hide differences in an existing definition.

MySQL DDL can commit independently. A multi-statement installation is not one rollbackable transaction. Preserve the error output if installation stops partway through.

```
USE hallsync_db;
SHOW TABLES;
SELECT COUNT(*) AS TableCount FROM information_schema.tables
WHERE table_schema = DATABASE() AND table_type = 'BASE TABLE';
```

# 2. Files and execution order

| File | Purpose and when to run |
|---|---|
| 01_create_database.sql | Already completed. Creates the empty project database. |
| 02_create_department.sql | Already completed. Creates the first table. |
| 03_insert_sample_department.sql | Already completed. Repeating its INSERT creates another department row. |
| complete_existing_database.sql | Recommended now. Combines 04, 05, 06, 07, 10 and 08. Run once on your existing one-table database. |
| fresh_install.sql | Alternative for a new empty installation. Requires the hallsync_db name to be unused. |
| 04_complete_schema.sql | Individual script: validates DEPARTMENT and adds the other 10 tables. |
| 05_business_workflows.sql | Individual script: installs public procedures and internal helpers. |
| 06_views_and_reports.sql | Individual script: creates four reporting views. |
| 07_demo_data.sql | Individual script: synthetic catalogue and SQL demonstration accounts; keeps existing matching records. |
| 08_verify_database.sql | Read-only structure and data-integrity checks; safe to run after setup. |
| 09_workflow_example.sql | Optional changing-data example: creates and approves a future demo Event request. |
| 10_application_role.sql | Creates a least-privilege MySQL role; does not create a login or password. |
| tests/run_checks.py | Developer validation. Creates isolated generated schemas, tests them, then removes only those schemas. |

Choose the combined continuation OR the individual 04-10 setup sequence. Once the combined file succeeds, use 08 to inspect it. The fresh-install file is for a different starting condition.

Every setup/business connection uses Asia/Colombo civil time through the +05:30 offset. This works without loading named MySQL timezone tables. Application connections must follow the same convention.

# 3. The design: nine entities become eleven tables

Each strong entity becomes a table. Each 1:N relationship stores the parent identifier in a child foreign-key column. Two M:N relationships need junction tables, bringing the total from nine entities to eleven tables.

| Table | Purpose |
|---|---|
| DEPARTMENT | Department catalogue shared by accounts, courses and groups. |
| BUILDING | Building codes and locations used to identify lecture halls. |
| FACILITY | Equipment types, such as Projector. Quantities belong to hall assignments. |
| USER_ACCOUNT | People, account activation, one effective role and password hashes. |
| COURSE_UNIT | Course catalogue and its owning department. |
| STUDENT_GROUP | Teaching groups by department and intake, not individual enrolment. |
| LECTURE_HALL | Physical hall identity, capacity and operational status. |
| BOOKING | One request, one responsible requester, one hall and at most one review decision. |
| HALL_BLOCK | Dated maintenance or other hall closures, including multi-day periods. |
| HALL_FACILITY | Hall-to-facility M:N assignment, with total and working quantities. |
| BOOKING_GROUP | Booking-to-student-group M:N assignment. |

Key legend: PRI = primary key, UNI = unique key, FK = foreign key. Required means NOT NULL.

A report is a query over existing data, so the design has no stored REPORT entity. The four views provide report datasets. SQL tables describe stored facts; OOP classes will also describe behaviour and do not have to map one-to-one to tables.

Names are not substituted for identifiers. For example, a booking stores RequesterID and HallID, while joins retrieve the current account and hall names.

# 4. Data dictionary: DEPARTMENT

Department catalogue shared by accounts, courses and groups.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| DepartmentID | int | Yes | PRI | Department identifier. Auto-generated in DEPARTMENT; a parent reference elsewhere. MySQL generates this primary-key value; gaps are allowed. |
| DepartmentName | varchar(100) | Yes | - | Required catalogue name. The CHECK rejects empty strings and ordinary-space-only names. Names are not a declared unique key. |

# 5. Data dictionary: BUILDING

Building codes and locations used to identify lecture halls.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| BuildingID | int | Yes | PRI | Building identifier; halls reference the building that contains them. MySQL generates this primary-key value; gaps are allowed. |
| BuildingCode | varchar(20) | Yes | UNI | Stable unique code used for building lookup. |
| BuildingName | varchar(100) | Yes | - | Required readable building name. |
| LocationDescription | varchar(255) | No | - | Optional description of the building location. |

# 6. Data dictionary: FACILITY

Equipment types, such as Projector. Quantities belong to hall assignments.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| FacilityID | int | Yes | PRI | Facility-type identifier. The same type can occur in several halls. MySQL generates this primary-key value; gaps are allowed. |
| FacilityName | varchar(100) | Yes | UNI | Unique type name, such as Projector. |
| Description | varchar(255) | No | - | Optional explanation of the facility type. |

# 7. Data dictionary: USER_ACCOUNT

People, account activation, one effective role and password hashes.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| AccountID | int | Yes | PRI | Account identifier. Reused as requester, reviewer or block recorder through differently named FKs. MySQL generates this primary-key value; gaps are allowed. |
| DepartmentID | int | No | FK | Department identifier. Auto-generated in DEPARTMENT; a parent reference elsewhere. |
| FullName | varchar(150) | Yes | - | Required display name. |
| Email | varchar(254) | Yes | UNI | Unique account email; application validates email format. |
| Username | varchar(50) | Yes | UNI | Unique login identifier; the database default collation makes lookup case-insensitive. |
| PasswordHash | varchar(255) | Yes | - | Encoded salted password hash. ASCII binary collation preserves its case-sensitive representation. |
| Role | enum('Student', 'Lecturer', 'AdminStaff') | Yes | - | One effective role: Student, Lecturer or AdminStaff. Privileged roles are provisioned by an administrator. Default: Student. |
| IsActive | BOOLEAN / TINYINT(1) | Yes | - | BOOLEAN alias for TINYINT(1). A CHECK restricts values to 0 or 1. Default: 1. |

PasswordHash stores no plaintext password. Hashing and password verification use a maintained Argon2id library in the application. The procedure checks the encoded prefix and length, not the original password or its strength.

# 8. Data dictionary: COURSE_UNIT

Course catalogue and its owning department.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| CourseID | int | Yes | PRI | Course identifier. Optional in a booking overall; mandatory for Lecture and Tutorial. MySQL generates this primary-key value; gaps are allowed. |
| DepartmentID | int | Yes | FK | Department identifier. Auto-generated in DEPARTMENT; a parent reference elsewhere. |
| CourseCode | varchar(20) | Yes | UNI | Unique catalogue code. |
| CourseTitle | varchar(150) | Yes | - | Required readable course title. |

# 9. Data dictionary: STUDENT_GROUP

Teaching groups by department and intake, not individual enrolment.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| GroupID | int | Yes | PRI | Student-group identifier. MySQL generates this primary-key value; gaps are allowed. |
| DepartmentID | int | Yes | FK | Department identifier. Auto-generated in DEPARTMENT; a parent reference elsewhere. |
| GroupName | varchar(100) | Yes | - | Teaching-group label; its department/intake/name combination is unique. |
| IntakeYear | smallint unsigned | Yes | - | Unsigned small integer, checked between 1900 and 9999. |
| StudyYear | tinyint unsigned | Yes | - | Unsigned tiny integer, checked to be at least 1. |

# 10. Data dictionary: LECTURE_HALL

Physical hall identity, capacity and operational status.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| HallID | int | Yes | PRI | Lecture-hall identifier. Every booking and hall block refers to one hall. MySQL generates this primary-key value; gaps are allowed. |
| BuildingID | int | Yes | FK | Building identifier; halls reference the building that contains them. |
| HallNo | varchar(20) | Yes | - | Room identifier, unique within its building rather than across all buildings. |
| Floor | varchar(10) | No | - | Optional text so values such as G or 1 can be represented. |
| Capacity | int | Yes | - | Positive integer capacity. Procedures compare attendance with this parent value. |
| OperationalStatus | enum('Active', 'Inactive') | Yes | - | Active or Inactive. Time availability is calculated from reservations and blocks. Default: Active. |

The hall has no permanent Booked flag. One hall can be reserved for one period and free for another. OperationalStatus describes the physical hall, while availability is calculated for the requested interval.

# 11. Data dictionary: BOOKING

One request, one responsible requester, one hall and at most one review decision.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| BookingID | int | Yes | PRI | Unique identifier for one request, including requests retained after cancellation or rejection. MySQL generates this primary-key value; gaps are allowed. |
| RequesterID | int | Yes | FK | Responsible account. Supplied from the authenticated session; there is no separate booked-for lecturer. |
| HallID | int | Yes | FK | Lecture-hall identifier. Every booking and hall block refers to one hall. |
| CourseID | int | No | FK | Course identifier. Optional in a booking overall; mandatory for Lecture and Tutorial. |
| BookingDate | date | Yes | - | Session date, interpreted with StartTime and EndTime in Asia/Colombo. |
| StartTime | time | Yes | - | Same-day start clock time. Procedures require the combined start instant to be in the future. |
| EndTime | time | Yes | - | Same-day end clock time, strictly after StartTime and before 24:00. |
| Purpose | varchar(255) | Yes | - | Required reason, limited to 255 characters. |
| BookingType | enum('Lecture', 'Tutorial', 'Exam', 'Event') | Yes | - | Lecture, Tutorial, Exam or Event. Academic types impose extra course/group requirements. |
| ExpectedAttendance | int | Yes | - | Positive integer expected headcount. Individual student enrolments are outside v1. |
| BookingStatus | enum('Pending', 'Approved', 'Rejected', 'Cancelled', 'Completed') | Yes | - | Current lifecycle state: Pending, Approved, Rejected, Cancelled or Completed. Default: Pending. |
| RequestedAt | datetime | Yes | - | Local date/time when the request was recorded. Default is CURRENT_TIMESTAMP. Default: CURRENT_TIMESTAMP. |
| ReviewerID | int | No | FK | Optional administrator reference; different from RequesterID when a review exists. |
| Decision | enum('Approved', 'Rejected') | No | - | Approved or Rejected. Preserve an earlier approval after later cancellation. |
| ReviewedAt | datetime | No | - | Optional local review timestamp; cannot be before RequestedAt. |
| ReviewNote | varchar(500) | No | - | Optional review explanation, up to 500 characters. |

A review is stored in this row because the agreed workflow allows at most one review decision. The reviewer ID, decision and review time are populated together. ReviewNote remains optional. The groups are separate BOOKING_GROUP rows.

# 12. Data dictionary: HALL_BLOCK

Dated maintenance or other hall closures, including multi-day periods.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| BlockID | int | Yes | PRI | Unique hall-closure identifier. MySQL generates this primary-key value; gaps are allowed. |
| HallID | int | Yes | FK | Lecture-hall identifier. Every booking and hall block refers to one hall. |
| RecordedByID | int | Yes | FK | Administrator who recorded the closure. |
| BlockStart | datetime | Yes | - | Closure start date/time; can be on a different date from BlockEnd. |
| BlockEnd | datetime | Yes | - | Closure end date/time, strictly later than BlockStart. |
| Reason | varchar(255) | Yes | - | Required explanation of the closure. |
| BlockStatus | enum('Active', 'Cancelled') | Yes | - | Active or Cancelled. Cancellation retains the closure record. Default: Active. |

# 13. Data dictionary: HALL_FACILITY

Hall-to-facility M:N assignment, with total and working quantities.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| HallID | int | Yes | PK part + FK | Lecture-hall identifier. Every booking and hall block refers to one hall. |
| FacilityID | int | Yes | PK part + FK | Facility-type identifier. The same type can occur in several halls. |
| Quantity | int | Yes | PK part + FK | Positive integer total of this facility type in this hall. |
| WorkingQuantity | int | Yes | PK part + FK | Integer between zero and Quantity inclusive; zero working units is valid. |

The two key columns are unique together. Either individual value can repeat in other rows. This represents an M:N relationship without a comma-separated list.

# 14. Data dictionary: BOOKING_GROUP

Booking-to-student-group M:N assignment.

| Column | SQL type | Required | Key | Meaning / reason |
|---|---|---|---|---|
| BookingID | int | Yes | PK part + FK | Unique identifier for one request, including requests retained after cancellation or rejection. |
| GroupID | int | Yes | PK part + FK | Student-group identifier. |

The two key columns are unique together. Either individual value can repeat in other rows. This represents an M:N relationship without a comma-separated list.

# 15. Primary keys, foreign keys and normalisation

| Child table.column | Parent table.column |
|---|---|
| BOOKING.CourseID | COURSE_UNIT.CourseID |
| BOOKING.HallID | LECTURE_HALL.HallID |
| BOOKING.RequesterID | USER_ACCOUNT.AccountID |
| BOOKING.ReviewerID | USER_ACCOUNT.AccountID |
| BOOKING_GROUP.BookingID | BOOKING.BookingID |
| BOOKING_GROUP.GroupID | STUDENT_GROUP.GroupID |
| COURSE_UNIT.DepartmentID | DEPARTMENT.DepartmentID |
| HALL_BLOCK.HallID | LECTURE_HALL.HallID |
| HALL_BLOCK.RecordedByID | USER_ACCOUNT.AccountID |
| HALL_FACILITY.FacilityID | FACILITY.FacilityID |
| HALL_FACILITY.HallID | LECTURE_HALL.HallID |
| LECTURE_HALL.BuildingID | BUILDING.BuildingID |
| STUDENT_GROUP.DepartmentID | DEPARTMENT.DepartmentID |
| USER_ACCOUNT.DepartmentID | DEPARTMENT.DepartmentID |

A foreign key checks that a referenced parent exists. It does not prove that an account is an administrator, that attendance fits the hall, or that two time intervals do not overlap. Those checks belong in the business procedures and authenticated application flow.

RequesterID and ReviewerID both reference USER_ACCOUNT.AccountID but describe different relationships. They are not new entities. A reviewer can review many bookings; an ordinary many-side FK must therefore not be made UNIQUE.

The junctions prevent repeated pairs. For example, one booking can serve groups 10 and 11, and group 10 can appear in other bookings. Only the duplicate pair is prohibited.

Parent records referenced by history use default InnoDB NO ACTION behaviour. Normal application permissions prohibit raw booking deletes. Deactivating a user or hall is different from deleting the referenced record.

# 16. Constraints and data types

| Choice | Reason |
| --- | --- |
| INT identifiers | Compact surrogate keys; generated IDs identify rows and are not record counts. |
| VARCHAR lengths | Bound the size of text values. The chosen limits are project design choices, not measured university limits. |
| DATE + TIME | Same-day bookings use a date and two clock times. Overnight bookings are outside this version. |
| DATETIME | Records request, review and hall-closure local timestamps. DATETIME itself stores no timezone identifier. |
| ENUM | Restricts the known role, status, type and decision vocabulary for this prototype. |
| NULL | Represents an optional or not-yet-known fact. It differs from an empty string and from zero. |
| NOT NULL + CHECK | NOT NULL rejects a missing value; a length check also rejects empty or ordinary-space-only text. |
| utf8mb4 | Unicode storage suitable for multilingual data. Connection encoding must also use utf8mb4. |
| ascii_bin PasswordHash | Encoded hashes are ASCII and case-sensitive, even though catalogue names use the database collation. |
| InnoDB | Supports foreign keys, row locks and transactions. |

TRIM in a CHECK is used to evaluate the condition; it does not rewrite the stored column. The public write procedures trim selected human-readable inputs before saving them.

The attendance API accepts a decimal parameter so it can reject a fractional value before inserting the required integer. Other raw form input, including email syntax and password policy, must also be validated by the application.

# 17. Row-level CHECK inventory

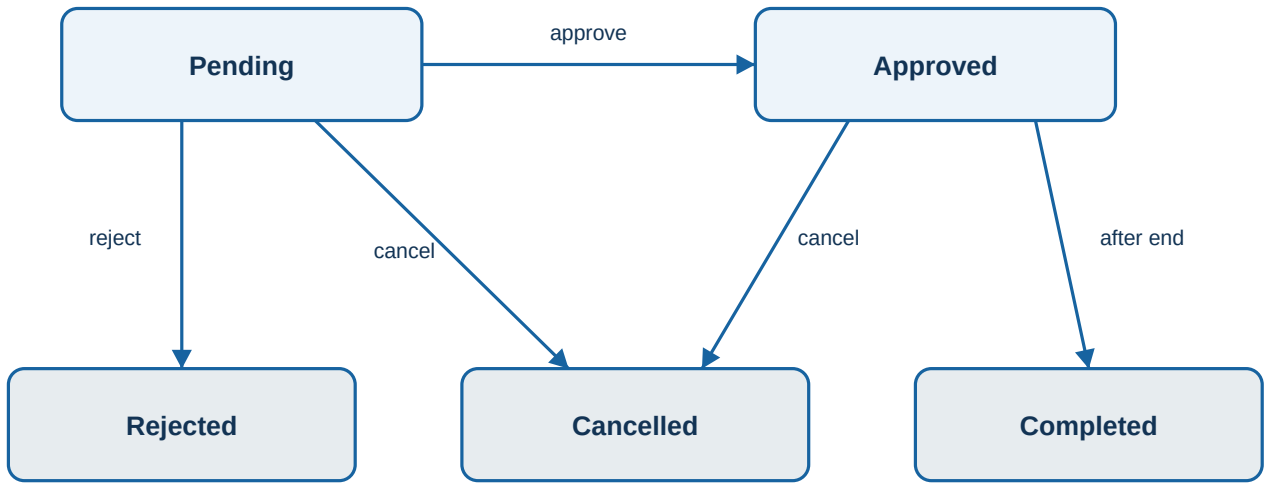
These 28 checks are taken from the validated MySQL schema. Required foreign-key links and uniqueness rules are additional constraints.

| Table / constraint | Expression |
|---|---|
| BOOKING / ck_booking_attendance | (`ExpectedAttendance` > 0) |
| BOOKING / ck_booking_clock | ((`StartTime` >= _utf8mb4\'00:00:00\') and (`StartTime` < _utf8mb4\'24:00:00\') and (`EndTime` >= _utf8mb4\'00:00:00\') and (`EndTime` < _utf8mb4\'24:00:00\') and (`EndTime` > `StartTime`)) |
| BOOKING / ck_booking_course | ((`BookingType` not in (_utf8mb4\'Lecture\',_utf8mb4\'Tutorial\')) or (`CourseID` is not null)) |
| BOOKING / ck_booking_no_self_review | ((`ReviewerID` is null) or (`ReviewerID` <> `RequesterID`)) |
| BOOKING / ck_booking_purpose | (char_length(trim(`Purpose`)) > 0) |
| BOOKING / ck_booking_review_bundle | (((`ReviewerID` is null) and (`Decision` is null) and (`ReviewedAt` is null) and (`ReviewNote` is null)) or ((`ReviewerID` is not null) and (`Decision` is not null) and (`ReviewedAt` is not null))) |
| BOOKING / ck_booking_review_state | (((`BookingStatus` = _utf8mb4\'Pending\') and (`ReviewerID` is null)) or ((`BookingStatus` in (_utf8mb4\'Approved\',_utf8mb4\'Completed\')) and (`ReviewerID` is not null) and (`Decision` = _utf8mb4\'Approved\')) or ((`BookingStatus` = _utf8mb4\'Rejected\') and (`ReviewerID` is not null) and (`Decision` = _utf8mb4\'Rejected\')) or ((`BookingStatus` = _utf8mb4\'Cancelled\') and ((`ReviewerID` is null) or (`Decision` = _utf8mb4\'Approved\')))) |
| BOOKING / ck_booking_review_time | ((`ReviewedAt` is null) or (`ReviewedAt` >= `RequestedAt`)) |
| BUILDING / ck_building_buildingcode | (char_length(trim(`BuildingCode`)) > 0) |
| BUILDING / ck_building_buildingname | (char_length(trim(`BuildingName`)) > 0) |
| COURSE_UNIT / ck_course_unit_coursecode | (char_length(trim(`CourseCode`)) > 0) |
| COURSE_UNIT / ck_course_unit_coursetitle | (char_length(trim(`CourseTitle`)) > 0) |
| DEPARTMENT / ck_department_departmentname | (char_length(trim(`DepartmentName`)) > 0) |
| FACILITY / ck_facility_facilityname | (char_length(trim(`FacilityName`)) > 0) |
| HALL_BLOCK / ck_hall_block_interval | (`BlockEnd` > `BlockStart`) |
| HALL_BLOCK / ck_hall_block_reason | (char_length(trim(`Reason`)) > 0) |
| HALL_FACILITY / ck_hall_facility_total | (`Quantity` > 0) |
| HALL_FACILITY / ck_hall_facility_working | ((`WorkingQuantity` >= 0) and (`WorkingQuantity` <= `Quantity`)) |
| LECTURE_HALL / ck_lecture_hall_capacity | (`Capacity` > 0) |
| LECTURE_HALL / ck_lecture_hall_hallno | (char_length(trim(`HallNo`)) > 0) |
| STUDENT_GROUP / ck_student_group_groupname | (char_length(trim(`GroupName`)) > 0) |

| Table / constraint | Expression |
|---|---|
| STUDENT_GROUP / ck_student_group_intake | (`IntakeYear` between 1900 and 9999) |
| STUDENT_GROUP / ck_student_group_level | (`StudyYear` >= 1) |
| USER_ACCOUNT / ck_user_account_active | (`IsActive` in (0,1)) |
| USER_ACCOUNT / ck_user_account_email | (char_length(trim(`Email`)) > 0) |
| USER_ACCOUNT / ck_user_account_fullname | (char_length(trim(`FullName`)) > 0) |
| USER_ACCOUNT / ck_user_account_passwordhash | (char_length(trim(`PasswordHash`)) > 0) |
| USER_ACCOUNT / ck_user_account_username | (char_length(trim(`Username`)) > 0) |

A permanent future-date CHECK is intentionally absent: an approved booking must remain valid history after its date passes. Future-start validation happens when creating, editing or approving a request.

# 18. Booking lifecycle and retained history




| State | Rules |
|---|---|
| Pending | Can be edited by its requester. Can be approved, rejected or cancelled. Does not reserve a slot. |
| Approved | Reserves the hall interval. Rescheduling requires cancellation and a new request. |
| Rejected | Terminal; retains its single rejection decision. |
| Cancelled | Terminal; retains a previous approval if it had one, along with group links. |
| Completed | Terminal; reached only from Approved after the session end. |

An authorised administrator can approve or reject another account request. Lecturers and authorised AdminStaff can submit their own requests; an administrator request still needs a different administrator to review it. Public registration creates only Student accounts.

The v1 schema preserves lifecycle records and one decision. It is not a complete audit log of every edit, cancellation actor or previous catalogue value.

# 19. Availability and overlapping intervals

```
newStart < existingEnd AND newEnd > existingStart
```

| Existing approved session | New request | Result |
|---|---|---|
| 09:00-10:00 | 09:30-10:30 | Conflict |
| 09:00-10:00 | 10:00-11:00 | Allowed adjacency |
| 09:00-10:00 | 08:00-09:00 | Allowed adjacency |
| 09:00-10:00 | 09:00-10:00 | Conflict |

The interval is half-open: the end of one session can equal the start of another. Booking comparisons use the same hall and booking date. For closures, TIMESTAMP(BookingDate, StartTime/EndTime) allows comparison with multi-day DATETIME block intervals.

An approval also requires an active hall, sufficient capacity, an active eligible requester and a future start instant. Lecture/Tutorial requires a course and at least one valid group.

Availability search is a snapshot. Another transaction can reserve the hall after the screen loads. Approval always rechecks the current data under the common hall lock.

Pending requests can overlap each other. Once one is approved, approving another overlapping request fails and leaves that second request Pending.

# 20. Transactions, locks and maintenance priority

- Begin a transaction using READ COMMITTED.

- Lock the relevant LECTURE_HALL row with SELECT ... FOR UPDATE.

- For an edit moving a Pending request, lock both halls in ascending HallID order.

- Lock the booking row, confirm its hall has not changed, and validate the actor and lifecycle state.

- Recheck future time, course/groups, capacity, approved bookings and active hall blocks.

- Write all related changes together, then COMMIT. On a SQL exception, ROLLBACK and return the error.

The common hall row is the coordination point. Booking approval and hall-block creation take the same lock. The second operation waits, then sees the first committed result before deciding whether it can proceed.

The selected READ COMMITTED isolation level avoids checking a snapshot created before waiting for the hall lock. Relevant hall-capacity and operational-status changes follow the same protocol.

A maintenance block cannot silently displace an approved booking. Conflicting future reservations must first be cancelled explicitly. A hall cannot become inactive or too small while that would invalidate an existing not-ended approved reservation.

Each public write procedure owns its transaction. Call it with application autocommit enabled and without an existing outer transaction. Handle SQLSTATE 45000 as a business-rule rejection; deadlocks and timeouts need rollback-aware retry. A network timeout after commit must not be blindly retried as a new request.

# 21. Public procedure reference

### sp_create_booking

Creates a Pending booking and group links atomically. The actor is the requester.

```
sp_create_booking(IN p_actor INT,IN p_hall INT,IN p_course INT,IN p_date DATE,IN p_start TIME,IN
    p_end TIME,IN p_purpose TEXT,IN p_type VARCHAR(20),IN p_attendance DECIMAL(20,6),IN p_groups
    JSON,OUT p_booking_id INT)
```

### sp_update_pending_booking

Owner-only Pending edit; can move halls using ordered locking. Replaces its group set atomically.

```
sp_update_pending_booking(IN p_actor INT,IN p_booking INT,IN p_hall INT,IN p_course INT,IN p_date
    DATE,IN p_start TIME,IN p_end TIME,IN p_purpose TEXT,IN p_type VARCHAR(20),IN p_attendance
    DECIMAL(20,6),IN p_groups JSON)
```

### sp_review_booking

Admin review of Pending; rechecks availability on approval and writes review fields together.

```
sp_review_booking(IN p_actor INT,IN p_booking INT,IN p_decision VARCHAR(10),IN p_note TEXT)
```

### sp_cancel_booking

Owner or admin cancels a future Pending/Approved request, preserving review and group history.

```
sp_cancel_booking(IN p_actor INT,IN p_booking INT)
```

### sp_complete_booking

Admin marks an ended Approved session Completed.

```
sp_complete_booking(IN p_actor INT,IN p_booking INT)
```

### sp_create_hall_block

Admin creates a dated closure after checking approved reservations under the hall lock.

```
sp_create_hall_block(IN p_actor INT,IN p_hall INT,IN p_start DATETIME,IN p_end DATETIME,IN p_reason
     TEXT,OUT p_block_id INT)
```

### sp_cancel_hall_block

Admin cancels an Active closure while retaining its row.

```
sp_cancel_hall_block(IN p_actor INT,IN p_block INT)
```

### sp_update_hall

Admin changes capacity/status only when approved reservations remain valid.

```
sp_update_hall(IN p_actor INT,IN p_hall INT,IN p_capacity INT,IN p_status VARCHAR(10))
```

### sp_set_hall_facility

Admin inserts or updates a hall/type pair with valid total and working quantities.

```
sp_set_hall_facility(IN p_actor INT,IN p_hall INT,IN p_facility INT,IN p_quantity INT,IN p_working
    INT)
```

### sp_register_student

Public registration contract. Role is fixed to Student; input is already a password hash.

```
sp_register_student(IN p_department INT,IN p_name TEXT,IN p_email TEXT,IN p_username TEXT,IN
    p_password_hash TEXT,OUT p_account_id INT)
```

### sp_provision_account

Administrator creates an account, including privileged lecturer/staff roles.

```
sp_provision_account(IN p_actor INT,IN p_department INT,IN p_name TEXT,IN p_email TEXT,IN
    p_username TEXT,IN p_password_hash TEXT,IN p_role VARCHAR(20),OUT p_account_id INT)
```

### sp_get_login_record

Backend-only username lookup for password verification; its hash must never reach public clients.

```
sp_get_login_record(IN p_username VARCHAR(50))
```

### sp_find_available_halls

Read-only future availability and capacity filter; it does not reserve a slot.

```
sp_find_available_halls(IN p_date DATE,IN p_start TIME,IN p_end TIME,IN p_attendance DECIMAL(20,6))
```

### sp_add_department

Admin adds a department catalogue record.

```
sp_add_department(IN p_actor INT,IN p_name VARCHAR(100),OUT p_id INT)
```

### sp_add_building

Admin adds a building catalogue record.

```
sp_add_building(IN p_actor INT,IN p_code VARCHAR(20),IN p_name VARCHAR(100),IN p_location
    VARCHAR(255),OUT p_id INT)
```

### sp_add_facility

Admin adds a facility-type catalogue record.

```
sp_add_facility(IN p_actor INT,IN p_name VARCHAR(100),IN p_description VARCHAR(255),OUT p_id INT)
```

### sp_add_course

Admin adds a course with an existing owning department.

```
sp_add_course(IN p_actor INT,IN p_department INT,IN p_code VARCHAR(20),IN p_title VARCHAR(150),OUT
    p_id INT)
```

### sp_add_student_group

Admin adds a teaching group with department/intake identity.

```
sp_add_student_group(IN p_actor INT,IN p_department INT,IN p_name VARCHAR(100),IN p_intake SMALLINT
     UNSIGNED,IN p_year TINYINT UNSIGNED,OUT p_id INT)
```

### sp_add_hall

Admin adds a physical hall in an existing building.

```
sp_add_hall(IN p_actor INT,IN p_building INT,IN p_number VARCHAR(20),IN p_floor VARCHAR(10),IN
    p_capacity INT,OUT p_id INT)
```

The six _hs_* helpers are implementation details: actor validation, hall locking, input validation, slot checks, group insertion and account-field validation. Application accounts receive EXECUTE on public procedures only, not helpers.

# 22. Run a complete booking example

After setup, the synthetic catalogue contains HS-DEMO / DEMO-01, a practice course, one student group and four demonstration accounts. The codes are invented for testing, not an official University of Kelaniya inventory.

```
SET @lecturer = (SELECT AccountID FROM USER_ACCOUNT WHERE Username='demo_lecturer');
SET @admin = (SELECT AccountID FROM USER_ACCOUNT WHERE Username='demo_admin_a');
SET @hall = (SELECT h.HallID FROM LECTURE_HALL h
 JOIN BUILDING b ON b.BuildingID=h.BuildingID
 WHERE b.BuildingCode='HS-DEMO' AND h.HallNo='DEMO-01');
SET @date = DATE_ADD(CURRENT_DATE(), INTERVAL 30 DAY);
CALL sp_create_booking(@lecturer,@hall,NULL,@date,'09:00','10:00',
 'Demo seminar','Event',30,JSON_ARRAY(),@booking);
CALL sp_review_booking(@admin,@booking,'Approved','Checked availability');
SELECT * FROM v_booking_details WHERE BookingID=@booking;
```

This Event does not need a course or student group. A Lecture/Tutorial passes an existing CourseID and JSON_ARRAY(group_id, ...). JSON is only an input format; the database stores one BOOKING_GROUP row per group.

Run database/09_workflow_example.sql once if you want this demonstration to create data. Repeating it can create another Pending request or fail because the earlier session is already approved.

To demonstrate cancellation later, call sp_cancel_booking with the requester or an administrator. The status becomes Cancelled while the earlier Approved decision remains recorded.

# 23. Views, reports and indexes

| View | Meaning |
|---|---|
| v_booking_details | One row per booking, joined with requester, hall/building, optional course and reviewer. |
| v_hall_facilities | Hall/type assignments with total, working and calculated non-working quantities. |
| v_booking_status_totals | Counts booking rows by their current state. |
| v_hall_usage | Counts approved/completed sessions and sums reserved minutes by hall/date. |

Joining a booking to multiple student groups can multiply result rows. Count bookings before joining, use an appropriate distinct count, or aggregate the junction separately. The provided status totals avoid this duplication.

Unique keys cover email, username, building code, facility name, course code, department/intake/group-name, and building/hall-number. The two junction primary keys also enforce unique pairs.

The booking interval index starts with HallID, BookingDate and BookingStatus. The block interval index starts with HallID and BlockStatus. These help find relevant rows; indexes alone do not forbid overlapping intervals.

Reserved minutes are scheduled session time, not actual measured hall use. The system does not track individual student attendance or enrolments.

# 24. Security and the application boundary

- The database validates roles for the supplied actor ID, but it cannot establish a human login session.

- The application must verify the password, check activation and derive actor IDs from that authenticated session.

- Never accept RequesterID or reviewer identity from an editable user form.

- Use a maintained Argon2id library. Validate password policy before hashing; the hash cannot reveal the original password length.

- The procedure hash-prefix check is a guard against obvious plaintext input, not proof that an encoded hash is genuine or uses adequate cost settings.

- Use prepared/parameterised calls in the application and a dedicated database login, not the setup administrator.

- Assign hallsync_application to that login. It permits approved interfaces and reads, while denying direct table writes and internal helper calls.

The demo hashes were generated from random discarded passwords. They enable SQL procedure demonstrations by account ID, but do not provide known login credentials. Provision real account credentials during the authentication lesson.

A privileged database administrator can bypass normal application rules through direct SQL or DDL. Operational users must use the restricted application interfaces. A shared database login is not an end-user identity system; production campus deployment should place database credentials in a trusted backend.

A first administrator and existing timetable data are controlled DBA bootstrap operations. Ordinary public registration can never choose Lecturer or AdminStaff.

# 25. Verification and test evidence

The read-only 08 script checks metadata and searches for conflicting approved bookings, approved/block overlaps, academic bookings without groups, and future reservations invalidated by hall capacity/status.

This package passed 100 checks on MySQL 8.0.46. The report is docs/database_validation.json. Tests create generated hallsync_validation_* schemas and never reset your hallsync_db.

| Category | What was checked |
|---|---|
| Structure | 11 tables, 61 columns, 11 PKs, 14 FKs, 7 other unique keys, 28 CHECKs, 4 views and 25 procedures. |
| Upgrade and fresh setup | Existing department ID/name preserved; demo seed repeats safely; fresh installation builds the same shape. |
| Integrity | Missing parents, duplicate equipment pairs, blank names, zero capacity and invalid equipment counts. |
| Business rules | Invalid times, fractional/zero/too-large attendance, course/group requirements, privileges and self-approval. |
| Lifecycle | Pending edits, rollback of invalid changes, cancellation history and terminal states. |
| Concurrency | 5 simultaneous conflicting-approval races, 3 approval/block races and a check that approval waits on the common hall lock. |
| Least privilege | Application login cannot insert raw bookings, read the raw password table or call internal helpers. |

## Passed check names

| Check | Result |
|---|---|
| Upgrade preserves existing department ID 1 | PASS |
| 11 tables | PASS |
| 61 columns | PASS |
| 11 PRIMARY KEY | PASS |
| 14 FOREIGN KEY | PASS |
| 7 UNIQUE | PASS |
| 28 CHECK | PASS |
| 4 views | PASS |
| 25 procedures | PASS |
| Demo seed can repeat without duplicate accounts | PASS |
| Blank department | PASS |
| NULL department | PASS |
| Missing parent | PASS |

| Check | Result |
|---|---|
| Zero capacity | PASS |
| Duplicate hall facility | PASS |
| Working exceeds total | PASS |
| Zero facility quantity | PASS |
| Student request | PASS |
| Attendance zero | PASS |
| Fractional attendance | PASS |
| Capacity overflow | PASS |
| Blank purpose | PASS |

| Check | Result |
|---|---|
| Equal times | PASS |
| Overnight times | PASS |
| Past start | PASS |
| Lecture missing course | PASS |
| Tutorial missing group | PASS |
| Unknown course | PASS |
| Unknown group | PASS |
| Duplicate groups | PASS |
| String group ID | PASS |
| Failed requests leave no booking rows | PASS |
| Overlapping Pending requests allowed | PASS |
| Group created with booking | PASS |
| Student cannot approve | PASS |
| Approval metadata consistent | PASS |
| Conflicting second approval | PASS |
| Failed review rolls back | PASS |
| Approved edit forbidden | PASS |
| Adjacent intervals allowed | PASS |
| Block cannot override approval | PASS |
| Capacity shrink cannot break reservation | PASS |
| Inactive cannot break reservation | PASS |
| Other requester cannot cancel | PASS |

| Check | Result |
|---|---|
| Cancellation retains approval | PASS |
| Cancellation retains groups | PASS |
| Cancelled terminal | PASS |
| Group history protects parent | PASS |
| Account history protects parent | PASS |
| Self approval prohibited | PASS |
| Different admin may approve staff request | PASS |
| Cannot complete before session end | PASS |
| Approval rechecks multi-day closure | PASS |
| Cancelled closure releases availability | PASS |
| Closure cancellation is terminal | PASS |
| Lecturer cannot create closure | PASS |
| Only owner edits Pending | PASS |
| Pending edit updates groups | PASS |
| Invalid edit rolls back | PASS |
| Old values retained after failed edit | PASS |
| Rejected terminal | PASS |
| Rejected cannot cancel | PASS |
| Pending cancellation needs no review | PASS |
| Ended approval can complete | PASS |
| Completed terminal | PASS |
| Student cannot provision lecturer | PASS |

| Check | Result |
|---|---|
| Plaintext password refused | PASS |
| Public registration fixes Student role | PASS |
| Admin provisions privileged role | PASS |
| Inactive requester refused | PASS |
| Working quantity zero allowed | PASS |
| Invalid inventory update | PASS |
| Student cannot manage inventory | PASS |
| Lecturer cannot add hall | PASS |
| Inactive hall refuses requests | PASS |
| Unreserved hall state can change | PASS |
| Details view has one row per booking | PASS |
| Status report totals match | PASS |
| Concurrent conflicting approvals 1 | PASS |
| Race leaves exactly one Approved 1 | PASS |
| Concurrent conflicting approvals 2 | PASS |
| Race leaves exactly one Approved 2 | PASS |
| Concurrent conflicting approvals 3 | PASS |
| Race leaves exactly one Approved 3 | PASS |
| Concurrent conflicting approvals 4 | PASS |
| Race leaves exactly one Approved 4 | PASS |
| Concurrent conflicting approvals 5 | PASS |
| Race leaves exactly one Approved 5 | PASS |

| Check | Result |
|---|---|
| Concurrent approval versus closure 1 | PASS |
| Concurrent approval versus closure 2 | PASS |
| Concurrent approval versus closure 3 | PASS |
| Approval waits for the common hall lock | PASS |
| Application cannot insert raw bookings | PASS |
| Application cannot read raw password table | PASS |
| Application cannot call internal helper | PASS |
| Application can read approved reporting interface | PASS |
| Application can call availability interface | PASS |
| Fresh installation creates 11 tables | PASS |
| Fresh installation creates 61 columns | PASS |
| No approved overlaps remain | PASS |

# 26. Timetable setup, scope and limitations

Availability is reliable only after existing future timetable reservations are entered with their responsible requester and a different authorised reviewer. Demo records are not a substitute for the real timetable. Controlled initial import is distinct from an ordinary user booking on behalf of someone else.

- No recurring booking series, automatic timetable generation, individual student enrolment tracking or multi-stage approvals.

- One booking uses one hall and one same-day session. Hall blocks can span multiple days.

- Faculty of Science is the fixed prototype context; there is no separate faculty catalogue.

- Reports are derived from stored facts. The model does not store a complete revision-by-revision audit log.

- Catalogue labels can change over time; the booking keeps identifiers and its own purpose/attendance, not historical snapshots of every parent field.

- Account credential verification, session handling, UI validation and user-facing messages are application work.

- No real university hall inventory, staff accounts or timetable has been imported by these scripts.

Store all system times consistently in Asia/Colombo. A start later today is valid when it is still in the future; an already started session cannot be submitted or approved as a new request.

# 27. Troubleshooting in Workbench

| Symptom | Action |
|---|---|
| Database/table already exists | You may have run a creation step already. Inspect with 08 and SHOW CREATE TABLE; preserve data instead of resetting. |
| Preflight expected only DEPARTMENT | The continuation starting condition differs, or a previous run stopped partway through. Share the exact error and existing table list. |
| Syntax error near DELIMITER | Open the SQL file and execute the whole procedure definition. DELIMITER is a client directive, not SQL to send as a server statement. |
| Procedure does not exist | Complete 05 first. Check the selected database and routine list. |
| Transaction characteristics cannot change | A write procedure was called within an existing transaction. Resolve that transaction and call the procedure outside an outer transaction. |
| CHECK / FK / duplicate-key error | Input violates a named rule. Fix the supplied values or create the parent record first. |
| SQLSTATE 45000 | Business-rule rejection. Display its message and leave the existing record unchanged. |
| Permission denied while saving a file | Use Save Script As in a writable normal folder. This file error is separate from database creation. |
| Password cannot log in | Demo passwords are intentionally unknown. Real credentials require application-side hashing/provisioning. |

Setup statements can partly succeed before an error. Retain Action Output and request a targeted repair. The scripts do not drop the project database or delete existing department rows.

# 28. Presentation questions and answers

### Why 11 tables for nine entities?

The two M:N relationships become HALL_FACILITY and BOOKING_GROUP junction tables.

### Why both NOT NULL and CHECK?

NULL and an empty string are different. A required text value also needs an appropriate non-empty rule.

### Why not store group IDs in one text column?

Separate junction rows support foreign keys, joins and pair uniqueness without parsing lists.

### Does a foreign key prove an administrator role?

It proves that the account exists. Role/activation checks are separate business logic.

### Why does Pending not reserve the hall?

Several requests may be reviewed; only an approved reservation guarantees that interval. Approval rechecks under a lock.

### Why use a transaction when approving?

The role/time/capacity/conflict checks and review/status update must succeed together or roll back.

### Why lock the hall row?

It exists even when no reservation exists yet, giving approvals and blocks a shared coordination point.

### Why can 10:00 follow a session ending 10:00?

The overlap test uses strict inequalities, so adjacent half-open intervals do not overlap.

### Why keep Decision after cancellation?

BookingStatus describes the current lifecycle, while Decision preserves the single earlier review.

### Is a foreign key OOP inheritance?

No. It is a relational reference. Classes, interfaces, encapsulation and polymorphism describe the application design.

### What will a BookingRepository do?

Expose save/search/review operations and translate object data into parameterised database calls.

### Where are password length rules checked?

Before hashing in the application. Only the salted encoded hash is stored.

# 29. Next: connect an OOP application

After you run the continuation script and confirm the 08 results, the next stage is the application model. Start with UserAccount, LectureHall and Booking responsibilities, then an interface such as BookingRepository and a database implementation.

A booking service coordinates authenticated actions. A repository calls the stored procedures and converts their results into objects. The UI presents information and explains rejections. Java can use JDBC with MySQL Connector/J; another chosen OOP language uses its supported MySQL driver.

Encapsulation protects object state, abstraction gives a clear repository contract, and polymorphism allows a test implementation of that contract. Use inheritance only for a justified is-a relationship. The database remains the authority for concurrent reservation decisions.

Your first application connection lesson will create a dedicated database login with a private password, assign hallsync_application, configure the timezone/encoding and test one read. No password belongs in a public repository.

# 30. Technical references

### MySQL CREATE TABLE and CHECK constraints

https://dev.mysql.com/doc/refman/8.0/en/create-table-check-constraints.html

### MySQL foreign keys

https://dev.mysql.com/doc/refman/8.0/en/create-table-foreign-keys.html

### MySQL locking reads

https://dev.mysql.com/doc/refman/8.0/en/innodb-locking-reads.html

### MySQL transaction isolation

https://dev.mysql.com/doc/refman/8.0/en/innodb-transaction-isolation-levels.html

### MySQL JSON_TABLE

https://dev.mysql.com/doc/refman/8.0/en/json-table-functions.html

### MySQL stored-object access control

https://dev.mysql.com/doc/refman/8.0/en/stored-objects-security.html

### OWASP Password Storage Cheat Sheet

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

### Java OOP learning material

https://dev.java/learn/language/oop/

### MySQL Connector/J

https://dev.mysql.com/doc/connector-j/en/connector-j-overview.html

Project specification: the agreed HallSync Chen-to-relational mapping, the discussed booking rules, and the SQL files in this repository. Synthetic catalogue values are explicitly labelled as demo data.